

Task Tracker API — Code Review Document

Project: Task Tracker API

Reviewer notes for: Internal code review session

Date: 2026-05-12

Branch: main

Language / Runtime: Node.js (v18+), Express 5

Lines of code: ~220 across application files

1. Executive Summary

The Task Tracker API is a small but cleanly structured RESTful service that lets a client create, read, update, and delete tasks. Each task is a simple object with three fields: a numeric `id`, a string `title`, and a boolean `completed` flag. The service does not persist data to a database — everything lives in memory in a JavaScript array — so all tasks vanish when the server restarts. That tradeoff is intentional for the scope of this module: the goal is to demonstrate mastery of Express routing, middleware, MVC layering, validation, and consistent error handling, not to ship a production data store.

The project follows the classic Node/Express MVC layout: `server.js` boots the app, `routes/` defines URL-to-handler mappings, `controllers/` holds the business logic, and `middleware/` contains cross-cutting concerns (logging, 404 handling, error handling). The code is short, readable, and consistent in style. There are no third-party packages beyond `express` and `dotenv`, which keeps the surface area small and the security footprint minimal.

Overall the codebase is in good shape for its intended scope. The remarks below are mostly about hardening, persistence, and operational concerns that would matter the moment this service crossed from "lab exercise" into "real product."

2. Project Layout — What Lives Where and Why

```
BEM-30 lab/
├─ server.js           ← Application entry point and middleware wiring
├─ package.json       ← Project metadata, dependencies, npm scripts
├─ routes/
│   └─ taskRoutes.js   ← Maps HTTP verbs + paths to controller functions
├─ controllers/
│   └─ taskController.js ← Business logic + in-memory data store
├─ middleware/
│   ├── logger.js      ← Logs every request when the response finishes
│   ├── notFound.js    ← Converts unknown routes into a 404 error
│   └─ errorHandler.js ← Central error responder
└─ README.md          ← User-facing API docs
```

The separation here is deliberate. Routes do nothing but wire URLs to functions — there is no logic in them. Controllers do not know anything about HTTP routing tables; they just receive `req` and `res`. Middleware sits orthogonal to both, handling concerns that apply across every endpoint. This makes each file small enough to understand in one read and easy to test in isolation.

3. The Application Entry Point — `server.js`

This file does five things, in this order:

1. **Loads environment variables.** The very first line is `require('dotenv').config()`, which reads the `.env` file and merges its contents into `process.env`. This must run before anything else, because later code reads `process.env.PORT`. If you reorder this line below the imports, the port lookup would happen against an unpopulated environment.
2. **Imports dependencies and local modules.** Express, the three middleware modules, and the task router.
3. **Builds the Express app and registers middleware in a specific order.** The order matters a great deal in Express, because middleware runs as a chain:
 - `express.json()` runs first so that `req.body` is parsed for every downstream handler.
 - `logger` runs next so that every request — including ones that hit invalid routes — gets logged.
 - The `GET / health` check is registered directly on the app.
 - The task router is mounted at `/api/tasks`.

- `notFound` runs after the router, so any request that didn't match a route falls through to it.
- `errorHandler` is registered last, because Express recognizes a middleware as an error handler only when it has the four-argument signature `(err, req, res, next)`, and only when it sits at the end of the chain.

4. **Starts the HTTP server.** `app.listen(PORT, ...)` returns a server instance that is captured in the `server` variable. Capturing it (rather than chaining `.listen()` directly) is what makes graceful shutdown possible later.
5. **Wires graceful shutdown.** The `gracefulShutdown` function is bound to both `SIGTERM` (sent by orchestration systems like Docker/Kubernetes when stopping a container) and `SIGINT` (Ctrl+C in a terminal). When either signal arrives, the server stops accepting new connections, waits for in-flight requests to finish, logs a message, and exits cleanly with code 0. Without this, the process would die abruptly mid-request.

Health endpoint. The `GET /` route returns `{ status: 'ok', service: 'task-tracker-api' }`. This is small but valuable: load balancers and uptime checkers can hit it to confirm the process is alive without needing to know any business endpoints.

4. Routing — `routes/taskRoutes.js`

This file is intentionally thin. It creates an Express `Router`, destructures the controller functions, and binds each HTTP method to a path:

Method	Path	Controller
GET	<code>/</code>	<code>getAllTasks</code>
GET	<code>/:id</code>	<code>getTaskById</code>
POST	<code>/</code>	<code>createTask</code>
PUT	<code>/:id</code>	<code>updateTask</code>
DELETE	<code>/:id</code>	<code>deleteTask</code>

Because the router is mounted at `/api/tasks` in `server.js`, the effective public paths are `/api/tasks`, `/api/tasks/:id`, and so on. Keeping the mount prefix in `server.js` (rather than baking `/api/tasks` into the route definitions) makes it trivial to version the API later — for example, mounting the same router at `/api/v1/tasks` would not require editing this file at all.

5. Business Logic — controllers/taskController.js

This file is the heart of the application. It holds the in-memory data store and the five CRUD handlers, plus two small helpers.

5.1 The In-Memory Store

```
let tasks = [];  
let nextId = 1;
```

`tasks` is an array of task objects. `nextId` is a monotonically increasing counter used to assign IDs when new tasks are created. Both are module-level `let` variables, which means they live for the lifetime of the Node process and are shared across every request — there is exactly one store, regardless of how many clients are connected.

Why this works for the lab: simplicity. No database driver, no migrations, no connection pool.

Why this is a limitation in production: the data is lost on restart, cannot be shared across multiple instances (so the service cannot be horizontally scaled), and the array scan is $O(n)$ on every lookup.

5.2 The Validation Helpers

Two helpers were extracted to remove duplication that previously existed across handlers:

`parseTaskId(rawId)` takes the raw string from `req.params.id` (Express always gives you strings for path parameters) and converts it to an integer. If the conversion produces `NaN` — which happens for anything non-numeric like `/api/tasks/abc` — it throws an error with `statusCode = 400` attached. The thrown error bubbles up to the global error handler, which formats the JSON response. This pattern (throwing an `Error` with a `statusCode` property) is the project's chosen convention for control flow on bad input.

`findTaskIndex(id)` locates a task by ID and returns the array index. If no task matches, it throws a 404 error. Notice it returns the **index**, not the task itself — that's because `updateTask` and `deleteTask` both need to mutate the array at a specific position, so an index is more useful than a reference.

`getTaskById` uses `Array.find` directly for the same reason inverted: it only needs the object, not the position.

5.3 getAllTasks

The simplest handler. It returns every task in the store along with a `count`. The response wraps the data in the project's standard envelope (`{ success, count, data }`). Including the count is convenient for clients that want to display "3 tasks" without having to look at `data.length`.

5.4 getTaskById

Parses the ID, finds the matching task with `Array.find`, throws a 404 if none is found, and returns the task. The duplicated 404-throwing block here (lines 36–40) could in principle be replaced with a call to `findTaskIndex`, but the current shape is slightly more direct because it doesn't need the index.

5.5 createTask

This handler does three things:

1. **Validates the title.** It must exist, must be a string, and must not be empty after trimming whitespace. The check `!title || typeof title !== 'string' || title.trim() === ''` covers all three cases. The order matters because of short-circuit evaluation — `!title` catches `undefined`, `null`, and empty string, but the `typeof` check is still needed to reject non-string values like numbers or booleans.
2. **Builds the task object.** The ID comes from `nextId++` (post-increment, so the task gets the current value and the counter advances). The title is trimmed to remove accidental whitespace. The `completed` field falls back to `false` if the client omits it or sends something non-boolean.
3. **Persists and responds.** Pushes onto the `tasks` array and returns a 201 with the new task.

The defensive `typeof completed === 'boolean'` check on the create path is worth highlighting — it means a request like `{ "title": "X", "completed": "true" }` (with the boolean stringified) silently coerces to `false` rather than rejecting. That's a deliberate choice favouring leniency on create. The update path, by contrast, rejects the same input with a 400. This asymmetry is intentional but worth noting in the review.

5.6 updateTask

The most complex handler. It supports partial updates — the client may send just `title`, just `completed`, both, or neither (a no-op). The logic:

1. Parse and validate the ID (throws 400 on bad input).
2. Locate the task by index (throws 404 on miss).
3. If `title` is present in the body, validate it and apply it.

4. If `completed` is present, validate that it's a boolean and apply it.
5. Return the updated task.

The use of `!== undefined` (rather than truthy checks) is important here. Without it, a request body like `{ "completed": false }` would be treated as "completed not provided" because `false` is falsy. Using `!== undefined` correctly distinguishes "the client sent the field with a falsy value" from "the client didn't send the field at all."

5.7 deleteTask

Parses the ID, locates the index, captures the task before removal, splices it out of the array, and returns the deleted task in the response. Returning the deleted record (rather than a bare success message) is a nice touch: clients can confirm what was removed and could even use the response to implement an "undo" feature.

6. Middleware Layer

6.1 logger.js

Each incoming request gets timed and logged. The logger captures `Date.now()` when the request enters, then attaches a `finish` listener to the response object. When Express finishes writing the response, the listener fires, computes the elapsed milliseconds, and prints a line like:

```
[2026-05-12T14:33:21.004Z] GET /api/tasks 200 12ms
```

Logging on `finish` (rather than at the start of the request) lets us capture the final status code and the duration in a single line. The downside is that requests that hang forever never log — but in practice that is a rare failure mode for this kind of service.

6.2 notFound.js

A thin middleware that runs after all routes. If a request reaches this point, no route matched, so it constructs an error like `Route not found: GET /api/unknown`, attaches a 404 status code, and calls `next(error)` to pass it to the error handler. Note that this uses the `next(err)` pattern rather than directly sending a response — keeping the response logic centralized in `errorHandler`.

6.3 errorHandler.js

The single point of truth for error responses. Any error — whether thrown synchronously in a controller, passed via `next(err)` from middleware, or surfaced by Express 5's built-in async handling — ends up here. The handler:

1. Reads `err.statusCode` if present, otherwise defaults to 500.
2. Decides what message to send back. In production, 500 errors are masked as a generic "Internal Server Error" so that internal details (file paths, stack traces, library internals) never leak to API clients. In non-production environments, the real error message is preserved.
3. Logs the error to the server console. In non-production environments, the full stack trace is also logged so developers can debug.
4. Sends the standard error envelope: `{ success: false, error: message } .`

The production/development split is driven by `NODE_ENV` . This is a small detail but important — it's the line between a debuggable dev experience and a safe production deployment.

7. Response Format

Every endpoint returns JSON in one of two shapes:

Success:

```
{ "success": true, "data": ..., "count": ?, "message": "..." }
```

Error:

```
{ "success": false, "error": "..." }
```

A single boolean `success` field at the top of every response makes client-side handling trivial: clients can branch on `response.success` before looking at anything else. This consistency is one of the strongest aspects of the project.

8. Configuration and Environment

A `.env` file controls runtime configuration. Currently only `PORT` is consumed, defaulting to 3000 if the variable is missing (`process.env.PORT || 3000`). Loading happens at startup via `dotenv` . This pattern is ready to absorb future secrets — database URLs, JWT keys, third-party API tokens — without changing how config is consumed.

9. Current Limitations

These are the honest gaps. Some are intentional for scope; some are areas to harden if this code ever moves toward production.

1. **No persistence.** All data is held in a JavaScript array. A restart, a crash, or a deploy wipes every task.
2. **No horizontal scaling.** Because the store is in-process, running two instances would give each one its own independent dataset. Load balancing across instances would produce incoherent behavior.
3. **No authentication or authorization.** Anyone who can reach the port can read, create, modify, or delete any task. There is no notion of "users" or "ownership."
4. **No rate limiting or abuse protection.** A client could hammer the API freely. There is no CORS configuration either — fine for local use but a gap for a real deployment.
5. **No automated tests.** There is no `tests/` directory, no Jest/Mocha setup, and `npm test` is not defined. All verification is manual via `cURL`/Postman.
6. **Limited input validation.** Title length is unbounded — a client could create a task with a 10MB title. The body-size cap inherited from `express.json()` (1MB default) is the only protection.
7. **`nextId` is monotonically increasing and never reused.** If a task with ID 5 is deleted, the next created task is still ID 6. This is correct behavior, but worth flagging because the IDs are not reused even after deletion, which can be surprising.
8. **Race conditions are not possible today** because Node is single-threaded for this synchronous logic — but if any controller becomes `async` and awaits between read and write, the in-memory store offers no locking.
9. **No structured logging.** The logger writes plain console strings, which is fine locally but hard to query in production (no JSON output, no correlation IDs, no log level).
10. **No request validation library.** Validation is hand-rolled inline. This works at five endpoints; it would become tedious and error-prone past a dozen.

11. **The 404 controller block in `getTaskById` duplicates `findTaskIndex`.** Minor cleanup opportunity — could be reduced to one helper call.
12. **Inconsistent leniency on `completed`.** Create silently coerces; update rejects. Pick one and document it.

10. Future Plans

In rough order of priority, the natural next steps are:

1. **Add a persistence layer.** The most obvious upgrade. A first step could be a JSON file on disk; a real step would be MongoDB or PostgreSQL. The controller's helper structure (`parseTaskId` , `findTaskIndex`) means the swap would mostly affect the data-access lines, not the response logic.
2. **Introduce a service layer.** Once persistence is real, controllers should not call the database directly. Inserting a `services/taskService.js` between controllers and the store keeps controllers focused on HTTP concerns and makes unit testing the business logic possible without spinning up Express.
3. **Add automated tests.** A Jest or Vitest suite covering each endpoint, each validation branch, and each error path. Supertest is the standard companion for HTTP-level testing of Express apps.
4. **Add authentication.** JWT-based auth is the typical first move. Each task would then belong to a user, and the controllers would filter by the authenticated user's ID.
5. **Replace hand-rolled validation with a schema library.** Zod or Joi would let each endpoint declare its expected body shape declaratively and reject mismatches uniformly. This also gives you free, accurate error messages.
6. **Add rate limiting, CORS, and Helmet.** Standard hardening middleware. `express-rate-limit` , `cors` , and `helmet` together cover most of the basic web-security checklist.
7. **Structured logging.** Replace `console.log` with `pino` or `winston` , emit JSON, include request IDs, and ship logs to a real aggregator.
8. **OpenAPI / Swagger documentation.** The README documents the API in prose; a machine-readable spec would let clients generate SDKs automatically.
9. **Containerization.** A Dockerfile plus a `docker-compose.yml` (once a database is involved) would make local setup one command.
10. **Pagination, filtering, and sorting on `GET /api/tasks`.** Once there are more than a handful of tasks, returning every record on every call is wasteful. Query parameters like `?completed=true&limit=20&offset=0` are the typical pattern.

11. Closing Notes

For a lab project sized around a single module, this code hits its targets cleanly: clear MVC layering, consistent response envelopes, centralized error handling, sensible middleware ordering, and graceful shutdown. The decisions to keep dependencies minimal and to push validation through `throw -with-statusCode` keep the codebase legible to a newcomer.

The most valuable next investments are persistence and tests, in that order — together they would lift this from "well-structured exercise" to "small but real production service." Everything else is incremental hardening on top of that foundation.